Wallet Payment Modeling Language - Part 2

Bianca-Maria Butica *up202603410@up.pt*
Mara-Liana Almasan *up202603436@up.pt*
Hanga Nemeth *up202603316@up.pt*
Oana-Maria Silaghi *up202603524@up.pt*

I. Introduction

Our problem is the *exact payment problem*, i.e. subset sum (class C): given a wallet of pieces and a target amount, find the pieces whose values add up to exactly the amount, optionally using at most `maxPieces` of them. The chain is

$$\text{Problem DSL} \xrightarrow{T1} \text{Solution DSL (linear integer constraints)} \xrightarrow{T2} \text{Quantum model} \xrightarrow{T3} \text{Qiskit}$$

T1 and T2 are Python transformations over BESSER (B-UML) models; T3 is BESSER's own Qiskit generator, used untouched. Everything is run by one command, `python pipeline.py`, which processes every file in `instances/`, executes the generated script on Aer and prints the payments found in domain terms (e.g. "pay with 5 euro cent coin + 2 euro cent coin + ..."). Solver choices (shots, number of Grover rounds, expected number of solutions) live in a separate solver configuration model (`config/`), not in the transformations.

II. Transformation

A. Realization

The solution is implemented as a sequence of three transformations. T1 converts a WPML problem model into a Solution DSL model containing linear constraints. T2 uses the constraints and the solver configuration to build a BESSER quantum circuit. T3 uses BESSER's Qiskit generator to produce a Python script, which the pipeline runs on Qiskit Aer. The pipeline then maps the measured bits back to pieces from the original wallet.

After installing the dependencies described in the README, the entire demonstration can be run from the repository root using the project's Python environment:

```
python pipeline.py
```

The command loads the five WPML problem specifications from `instances/`, generates and runs the Qiskit scripts, and prints the pieces found for each valid payment. The generated scripts are stored under `output/`, and the comparison with the classical baseline is saved in `output/validation.md`.

B. Content

T1. A payment becomes a constraint system, and each piece in its wallet becomes a one-bit variable y_i. The variable is 1 if the piece is selected and 0 otherwise. For a piece of value v_i and a target amount A, the exact-payment condition is

$$\sum_i v_i y_i - A = 0.$$

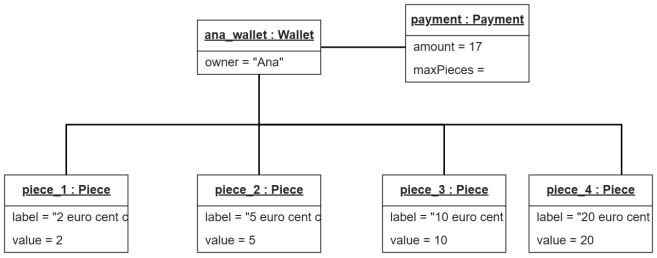


Fig. 1. T1 input: Ana's wallet (2, 5, 10 and 20 cents) and a payment of 17 cents, as a WPML object model.

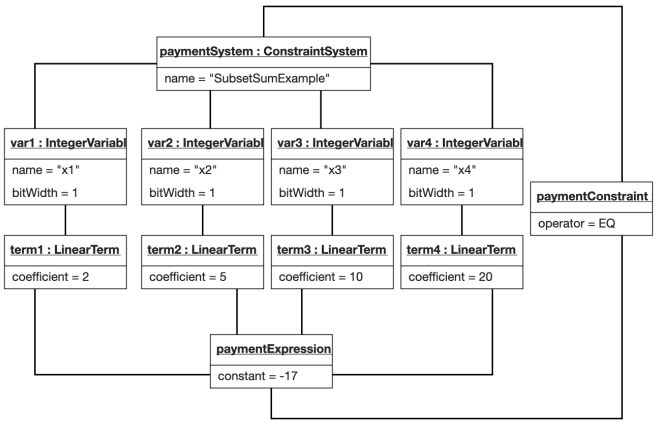


Fig. 2. T1 output for the same payment, as a Solution DSL object model: $-17 + 2x_1 + 5x_2 + 10x_3 + 20x_4 = 0$ (built by hand in Part I; T1 produces the same structure, with variables named after the pieces).

This follows the subset-sum formulation in Annex A, Class C from the assignment specifications. If the payment has a limit of k pieces, T1 also adds $\sum_i y_i - k \leq 0$ and combines the two constraints with AND. T1 records which piece each variable represents so the payment can be reconstructed after measurement.

T2. The circuit uses search qubits to represent choices, a phase qubit to mark valid choices, and auxiliary qubits for arithmetic and constraint checks. T2 calculates the range of each expression and allocates enough bits to store its signed value. It uses controlled X gates to add the selected values reversibly [1], [5]. An equality is checked by testing whether the expression e is zero. For $e \leq 0$, the circuit computes $e - 1$ and checks its sign.

The Grover oracle evaluates the constraints, marks valid choices through a phase change and reverses its calculations

to clear the auxiliary qubits. The diffuser then increases the probability of measuring valid choices [3]. The solver configuration allows a fixed number of Grover rounds. When a round count is omitted, the pipeline uses the number of valid subsets found by classical enumeration to choose the rounds [2]. The default configuration uses 1,024 measurements per instance.

T3. BESSER generates a Qiskit script from the circuit model. Once the script runs, the pipeline uses the mappings from T1 and T2 to translate a measured bitstring into the labels of the selected pieces. This is how the program reports a payment in terms of the original wallet.

III. Validation

The classical baseline enumerates subsets of pieces and checks their total value and the optional piece limit. For each instance, the validation also checks the reversible arithmetic and oracle on every candidate input. It verifies that the expected candidates are marked and that the auxiliary bits are cleared. The set of valid payments observed in the simulation agreed with the classical baseline in all five cases [4].

			Oracle	P(valid)	
Instance	Qubits	M	calls	measured	theory
`complete_model`	12	1	3	95.9%	96.1%
`limit`	17	3	3	99.7%	99.8%
`many`	15	3	2	100.0%	100.0%
`none`	10	0	0	0.0%	0.0%
`tiny`	11	1	3	96.5%	96.1%

M is the number of valid payments found by the baseline, the oracle calls are the Grover rounds per shot, and the theory column is $\sin^2((2k+1)\theta)$ with $\sin^2\theta = M/N$ [2].

A. Limitations.

- M **is taken from the classical baseline.** The number of Grover rounds is only known because we solve the problem classically first. A real use would need the unknown-M variant (Boyer et al., 1998) [2], which we did not implement.
- **Qubit growth.** Besides one search qubit per piece, each expression needs an arithmetic register and ancillas, so we reach about 17 qubits with a modest wallet. Larger wallets quickly become too large to simulate, since each extra piece doubles the number of choices.
- **No quantum advantage.** Dynamic programming solves these instances in pseudo-polynomial time, and the simulator is slower than classical code.
- **Scope.** Our problem only produces 1-bit unknowns (each piece used once) and linear constraints, with no products of variables; a limit on the number of pieces is the only extra constraint. We only ran on a simulator, not on real hardware.
- **Decoding outside the generated script.** The generated `qiskit_circuit.py` only prints the raw measurement counts; the translation into pieces is done by `pipeline.py`, using the T1 and T2 mappings.

References

[1] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter. Elementary gates for quantum computation. *Physical Review A*, 52(5):3457–3467, 1995.

[2] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. Tight bounds on quantum searching. *Fortschritte der Physik*, 46(4-5):493–505, 1998.

[3] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219, 1996.

[4] TACS Project Repository. Recorded validation results. https://github.com/biencutica/tacs26/blob/e7897d5/output/validation.md, 2026. `branch/mara` at `e7897d5`.

[5] V. Vedral, A. Barenco, and A. Ekert. Quantum networks for elementary arithmetic operations. *Physical Review A*, 54(1):147–153, 1996.